Experiment 8: Deploying Applications, CI/CD Pipeline, and Cluster Monitoring

Name: Manish Raje
Course Outcomes: CO3, CO5, CO6
Environment: Windows 11, Docker Desktop 28.3.2, Minikube v1.39.0 (Kubernetes v1.37.0), kubectl v1.32.2, Helm v4.3.1

1. GitHub Repository

https://github.com/manishraje28/exp8-k8s-cicd

Docker Hub image: https://hub.docker.com/r/manishraje28/sample-web-app

2. Pre-Lab Diagnostic Quiz

Q1. What is the structural relationship between a Deployment, a ReplicaSet and a Pod?

A Deployment is the top-level declarative object. It creates and manages ReplicaSets, one per revision of the Pod template. Each ReplicaSet keeps the requested number of identical Pods running and replaces any that fail. When the Deployment changes (for example a new image), it creates a new ReplicaSet and scales it up while scaling the old one down, which gives rolling updates and rollback. In this lab, sample-web-app first owned ReplicaSet 5cf58b94f4 (nginx:alpine); after the image update a new ReplicaSet 64dbf556ff (the CI-built image) replaced it.

Q2. Why should Docker Hub credentials never be stored in .github/workflows/ci-cd.yml? Where should they be stored?

The workflow file is committed to the repository, so anything in it is visible to anyone who can read the repo and stays in Git history even after it is removed. A leaked token lets anyone push or overwrite images. Credentials belong in GitHub encrypted Repository Secrets (Settings > Secrets and variables > Actions). The workflow reads them as ${{ secrets.DOCKERHUB_USERNAME }} and ${{ secrets.DOCKERHUB_TOKEN }}, and GitHub masks them in logs. A scoped Docker Hub access token is used instead of the account password so it can be revoked on its own.

Q3. What component collects CPU and memory metrics from kubelets so that kubectl top pods works?

The Kubernetes Metrics Server. It scrapes resource usage from each node's kubelet and exposes it through the metrics.k8s.io API, which kubectl top and the Horizontal Pod Autoscaler use. In Minikube it is enabled with "minikube addons enable metrics-server".

3. Environment Setup

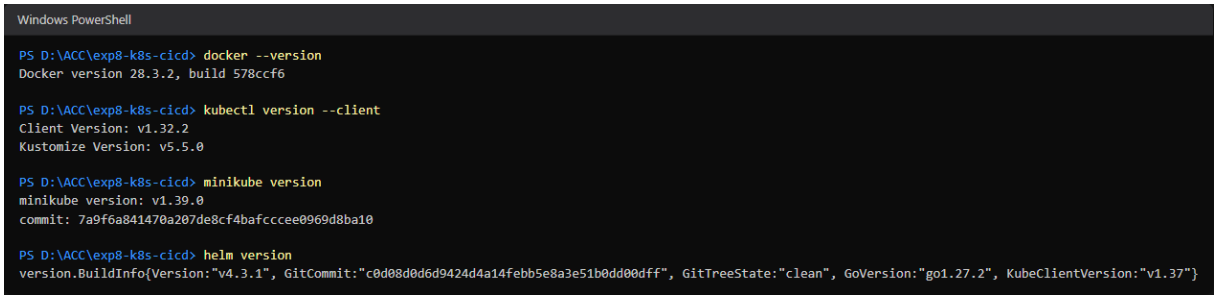


Fig 1. Installed tool versions



Fig 2. Minikube cluster running, node Ready

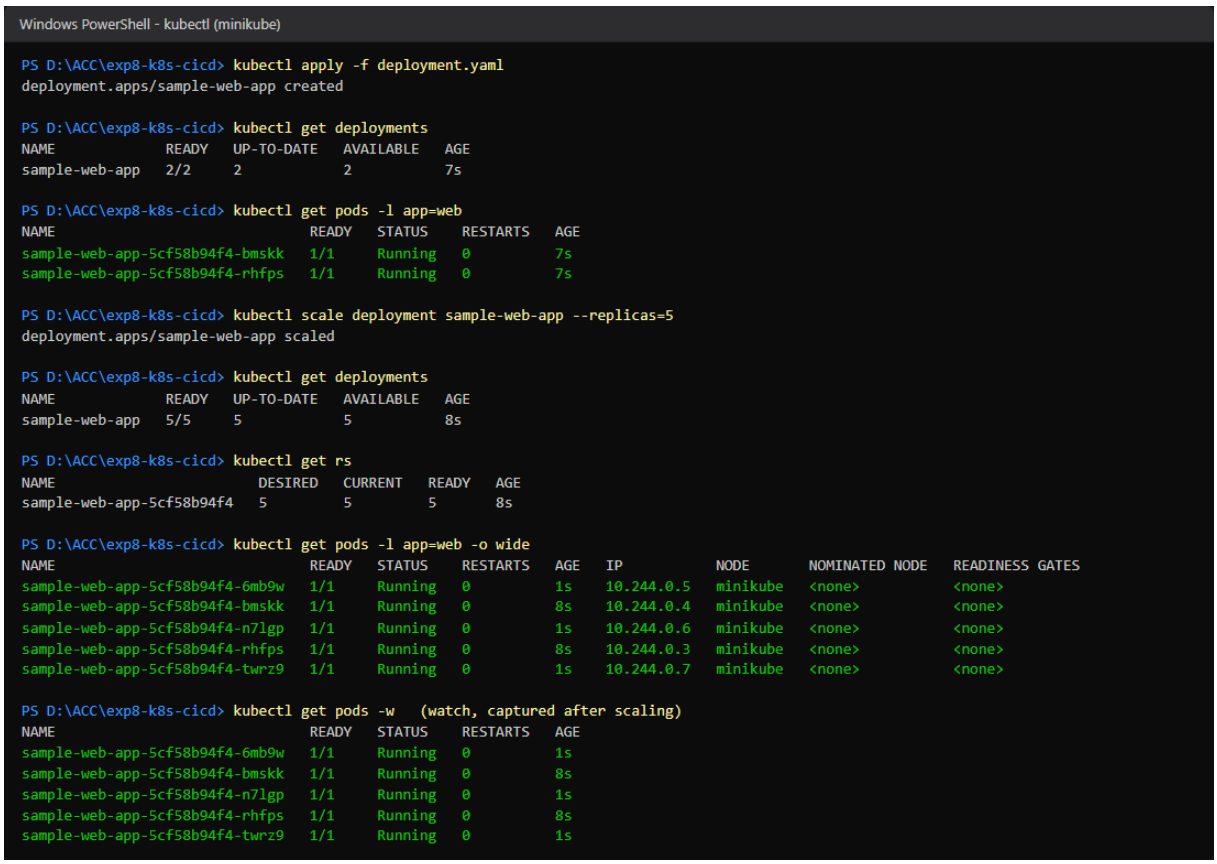
4. Activity 1: Deployment and Replica Scaling (CO3)

deployment.yaml:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: sample-web-app
  labels:
    app: web
spec:
  replicas: 2
  selector:
    matchLabels:
      app: web
  template:
```

```
  metadata:
    labels:
      app: web
  spec:
    containers:
      - name: web-container
        image: nginx:alpine
        ports:
          - containerPort: 80
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "250m"
            memory: "256Mi"
```

Commands: kubectl apply -f deployment.yaml; kubectl get deployments; kubectl get pods -l app=web; kubectl scale deployment sample-web-app --replicas=5; kubectl get pods -w



Screenshot 1. kubectl get pods showing 5 active scaled pods

5. Activity 2: CI/CD Pipeline with GitHub Actions (CO5)

Repository secrets DOCKERHUB_USERNAME and DOCKERHUB_TOKEN were added under Settings > Secrets and variables > Actions.

Dockerfile:

```
FROM nginx:alpine
COPY app/index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

.github/workflows/ci-cd.yml:

```
name: CI/CD Pipeline - Build and Push
on:
  push:
    branches: [ "main" ]
jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Source Code
        uses: actions/checkout@v3
      - name: Log in to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v2
      - name: Build and Push Docker Image
        uses: docker/build-push-action@v4
        with:
          context: .
          push: true
          tags: ${{ secrets.DOCKERHUB_USERNAME }}/sample-web-app:v1.0.${{
github.run_number }}
```

```
Windows PowerShell - git

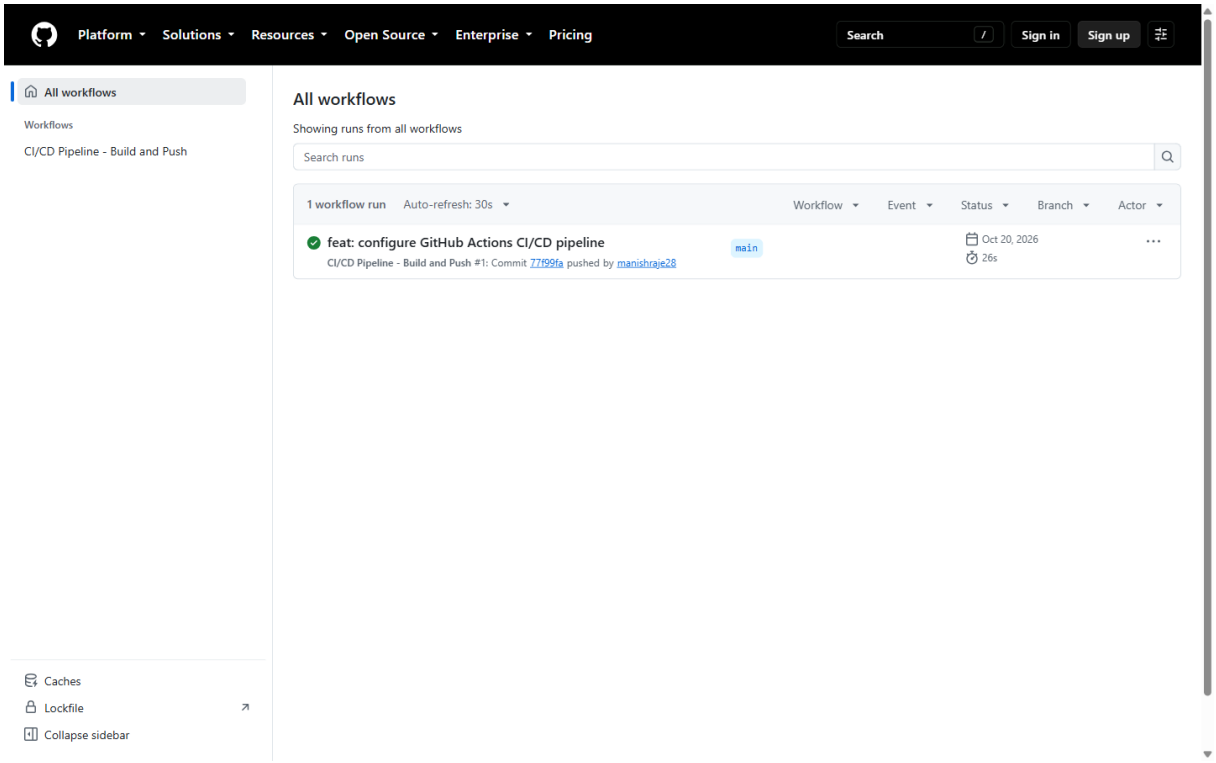
PS D:\ACC\exp8-k8s-cicd> git add .

PS D:\ACC\exp8-k8s-cicd> git commit -m "feat: configure GitHub Actions CI/CD pipeline"
[main 77f99fa] feat: configure GitHub Actions CI/CD pipeline

 .github/workflows/ci-cd.yml               |  23 ++++++++
 .gitignore                                |   4 ++
 Dockerfile                                |   3 ++
 Experiment8_Step_Log.docx                 | Bin 0 -> 941786 bytes
 app/index.html                            |   6 +++
 build_steplog.py                          |  13 +++++
 deployment.yaml                           |  28 ++++++++++
 logs/00_setup_tools.txt                   |  13 +++++
 logs/01_minikube.txt                      |  18 +++++++
 logs/02_files.txt                         |  36 ++++++++++++
 logs/03_workflow.txt                      |  29 ++++++++++
 logs/activity1.txt                        |  38 ++++++++++++
 logs/activity3a.txt                       |  29 ++++++++++
 logs/activity3b.txt                       |  47 +++++++++++++++
 logs/dashboard_url.txt                    |   4 ++
 logs/docker_local.txt                     |  83 +++++++++++++++++++++++++
 screenshots/00_setup_tools.png            | Bin 0 -> 28463 bytes
 screenshots/01_minikube.png               | Bin 0 -> 30389 bytes

PS D:\ACC\exp8-k8s-cicd> git push origin main
branch 'main' set up to track 'origin/main'.
To https://github.com/manishraje28/exp8-k8s-cicd.git
 * [new branch]      main -> main
```

Fig. git commit and push to main



Screenshot 2. Completed green run in GitHub Actions workflow history

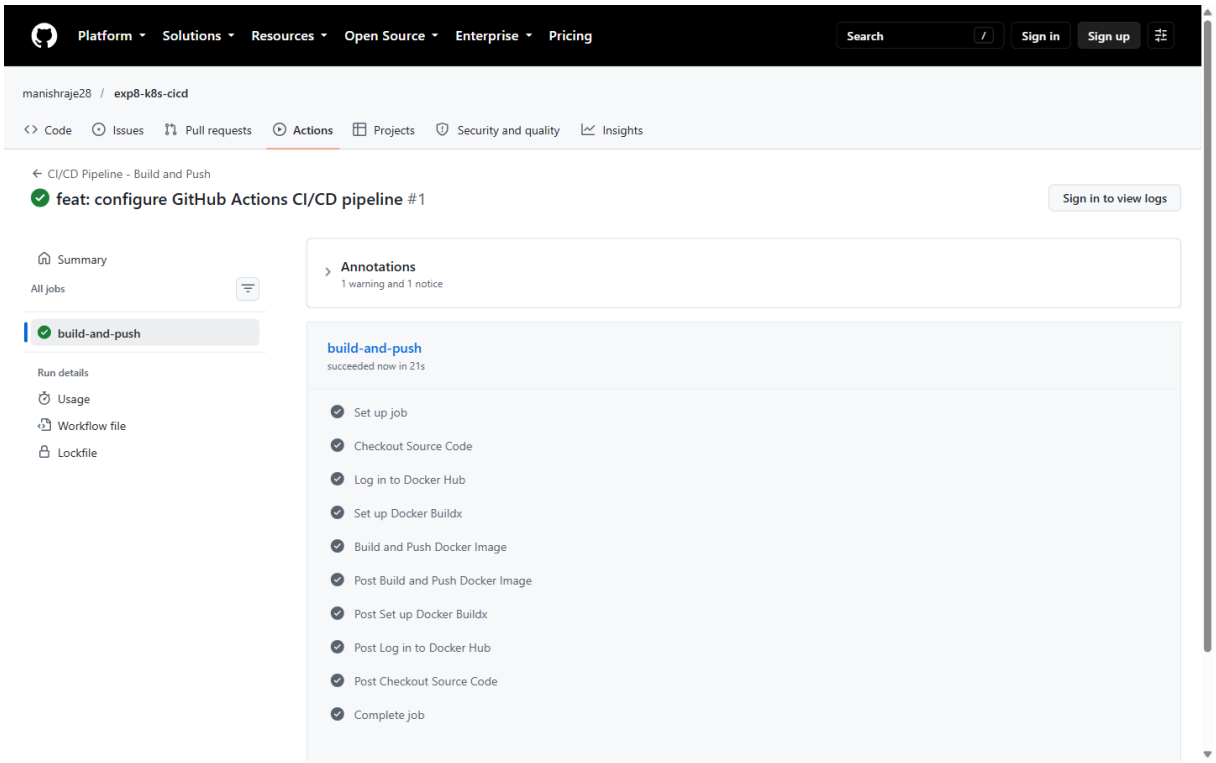


Fig. Pipeline log: every build-and-push job step succeeded (run #1)

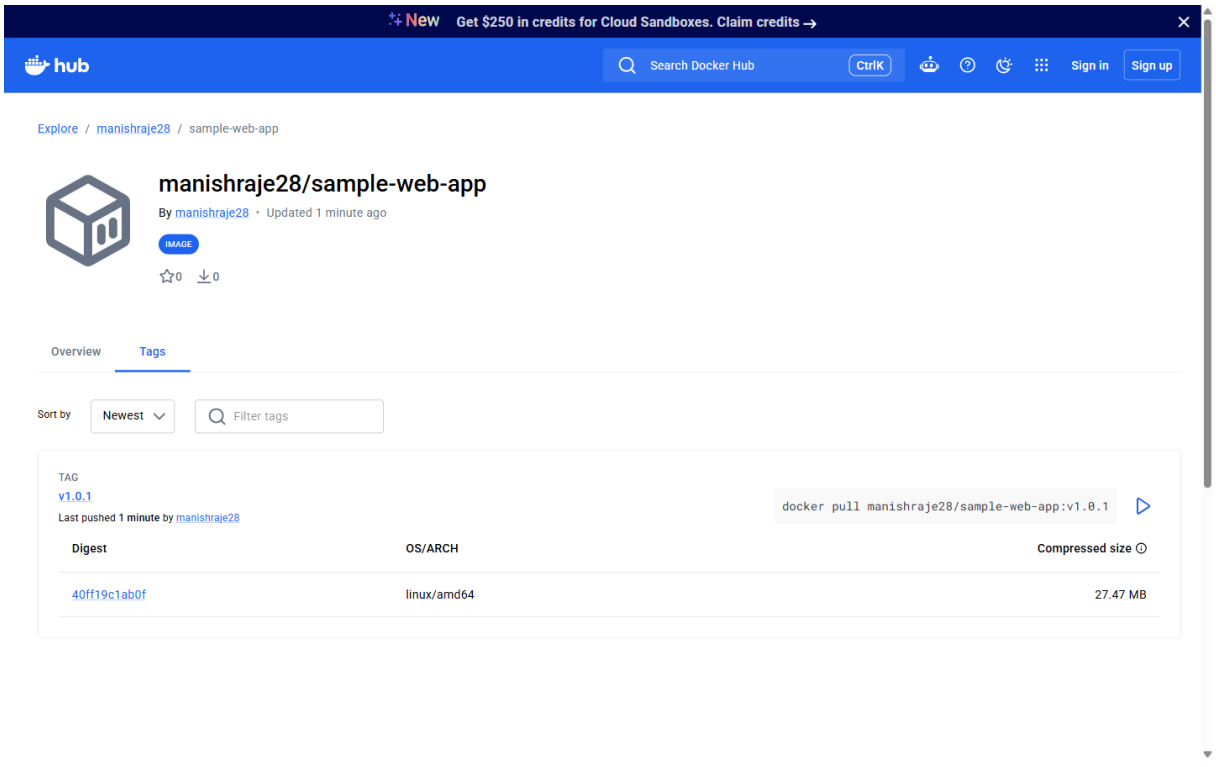


Fig. Image manishraje28/sample-web-app:v1.0.1 delivered to Docker Hub

```
Windows PowerShell - kubectl

PS D:\ACC\exp8-k8s-cicd> kubectl set image deployment/sample-web-app web-container=manishraje28/sample-web-app:v1.0.1
deployment.apps/sample-web-app image updated

PS D:\ACC\exp8-k8s-cicd> kubectl rollout status deployment/sample-web-app --timeout=180s
Waiting for deployment "sample-web-app" rollout to finish: 3 out of 5 new replicas have been updated...
Waiting for deployment "sample-web-app" rollout to finish: 4 of 5 updated replicas are available...
deployment "sample-web-app" successfully rolled out

PS D:\ACC\exp8-k8s-cicd> kubectl exec deploy/sample-web-app -- wget -qO- localhost | findstr h1
<h1>Experiment 8: Kubernetes + CI/CD</h1>

PS D:\ACC\exp8-k8s-cicd> kubectl get pods -l app=web -o custom-columns=NAME:.metadata.name,STATUS:.status.phase,IMAGE:.spec.containers[0].image
NAME                          STATUS    IMAGE
sample-web-app-64dbf556ff-52g2n   Running   manishraje28/sample-web-app:v1.0.1
sample-web-app-64dbf556ff-lr8x8   Running   manishraje28/sample-web-app:v1.0.1
sample-web-app-64dbf556ff-pfxcg   Running   manishraje28/sample-web-app:v1.0.1
sample-web-app-64dbf556ff-sbzzh   Running   manishraje28/sample-web-app:v1.0.1
sample-web-app-64dbf556ff-vxlmc   Running   manishraje28/sample-web-app:v1.0.1
```

Fig. CI-built image rolled out to all 5 replicas on Minikube

6. Activity 3: Cluster and Pod Monitoring (CO6)

Both Option A (Minikube Dashboard) and Option B (Prometheus and Grafana via Helm) were completed.

```
Windows PowerShell - metrics-server

PS D:\ACC\exp8-k8s-cicd> minikube addons enable metrics-server
* metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  - Using image registry.k8s.io/metrics-server/metrics-server:v0.9.0
* The 'metrics-server' addon is enabled

PS D:\ACC\exp8-k8s-cicd> minikube addons enable dashboard
* dashboard is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
  - Using image docker.io/kubernetesui/dashboard:v2.7.0
  - Using image docker.io/kubernetesui/metrics-scraper:v1.0.8
* Some dashboard features require the metrics-server addon. To enable all features please run:

        minikube addons enable metrics-server

* The 'dashboard' addon is enabled

PS D:\ACC\exp8-k8s-cicd> kubectl top nodes
NAME       CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
minikube   311m         1%       758Mi           9%

PS D:\ACC\exp8-k8s-cicd> kubectl top pods
NAME                              CPU(cores)   MEMORY(bytes)
sample-web-app-5cf58b94f4-6mb9w   0m           15Mi
sample-web-app-5cf58b94f4-bmskk   0m           15Mi
sample-web-app-5cf58b94f4-n7lgp   0m           15Mi
sample-web-app-5cf58b94f4-rhfps   0m           15Mi
sample-web-app-5cf58b94f4-twrz9   0m           15Mi
```

Fig. metrics-server enabled; kubectl top nodes and pods

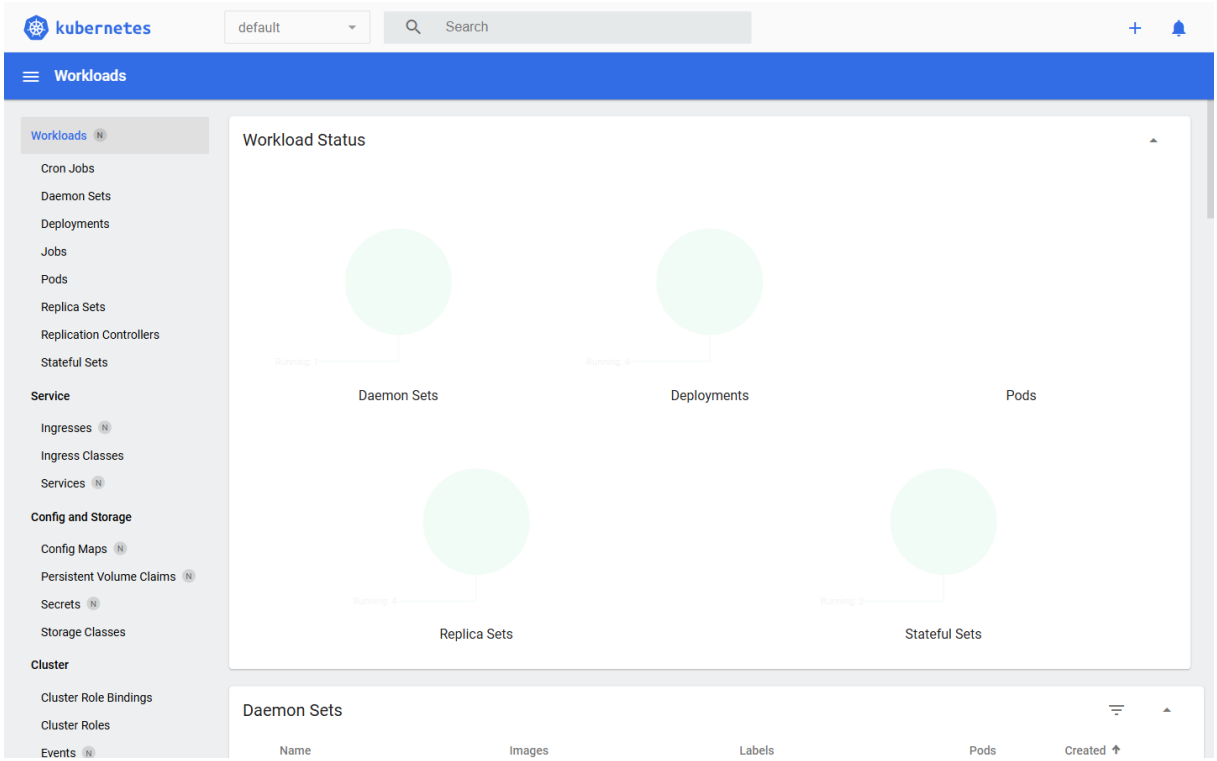


Fig. Minikube Kubernetes Dashboard: sample-web-app 5/5 pods

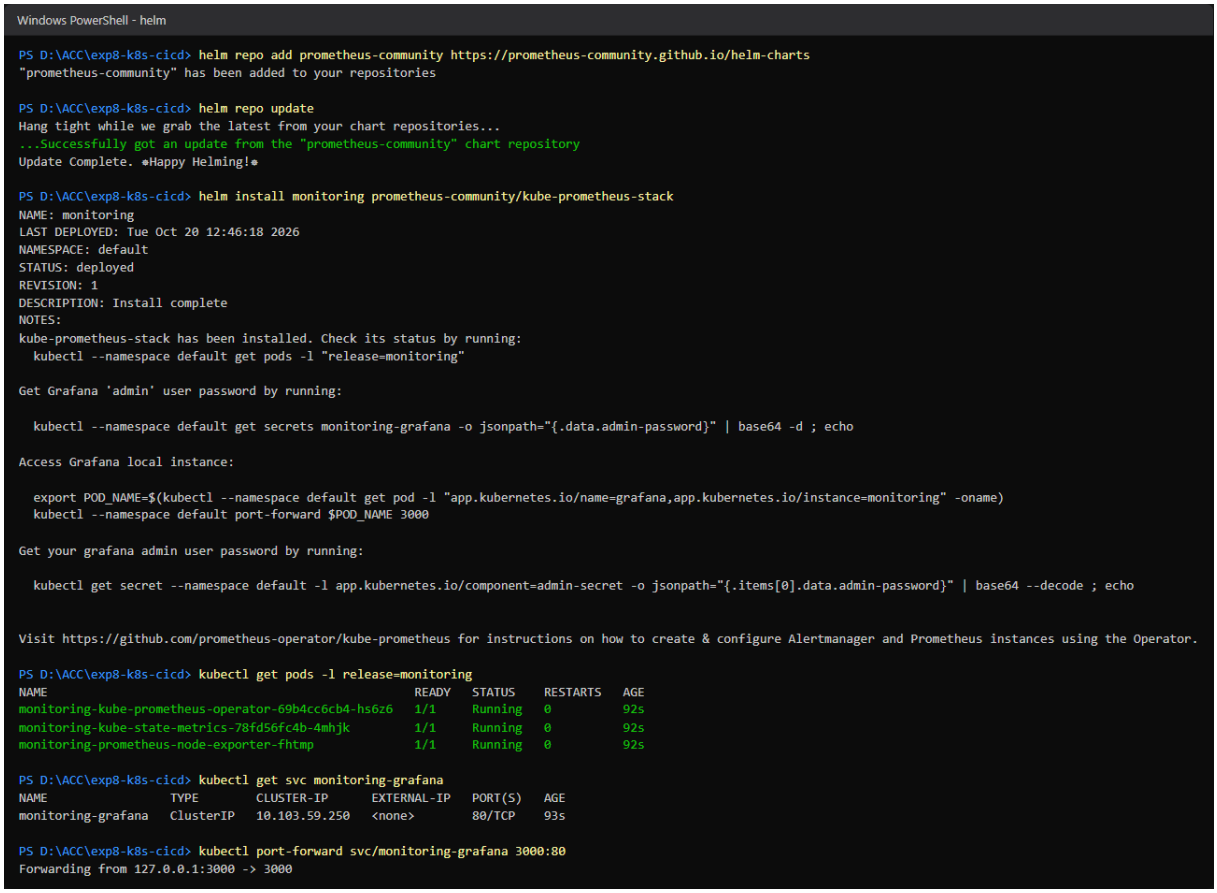
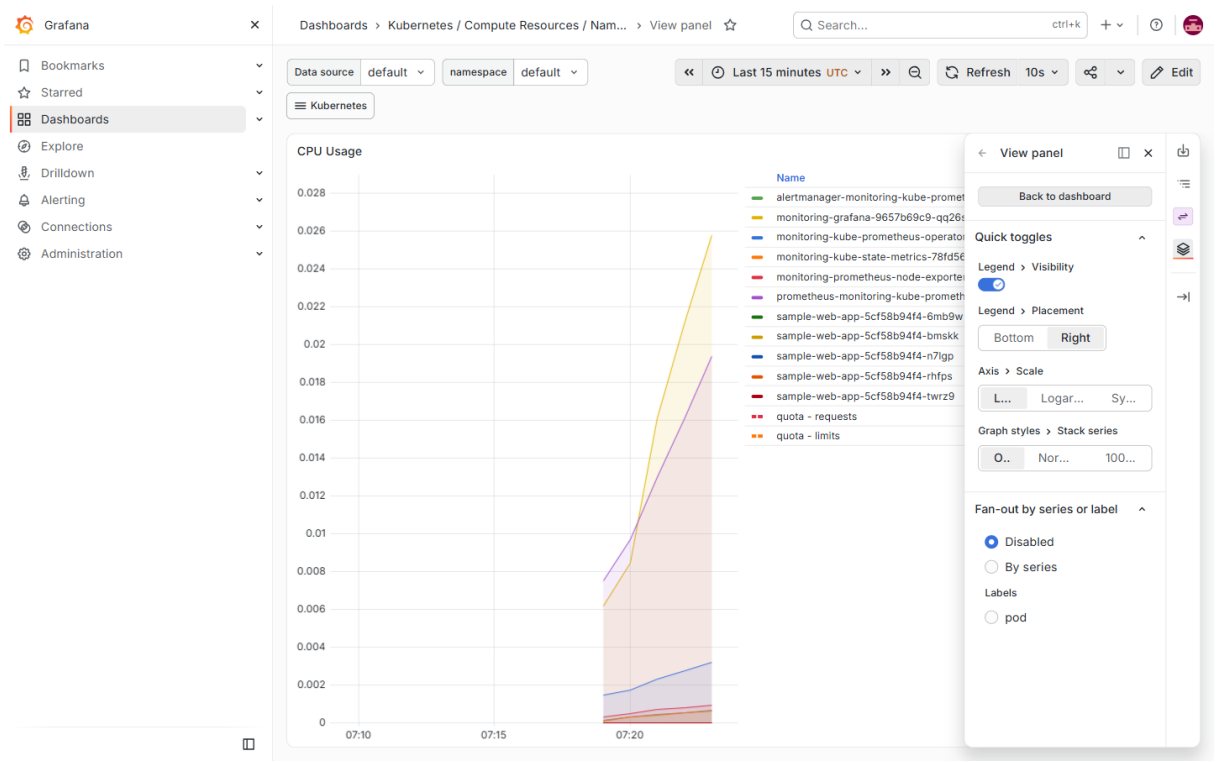


Fig. kube-prometheus-stack installed with Helm



Screenshot 3. Live CPU usage per pod in Grafana

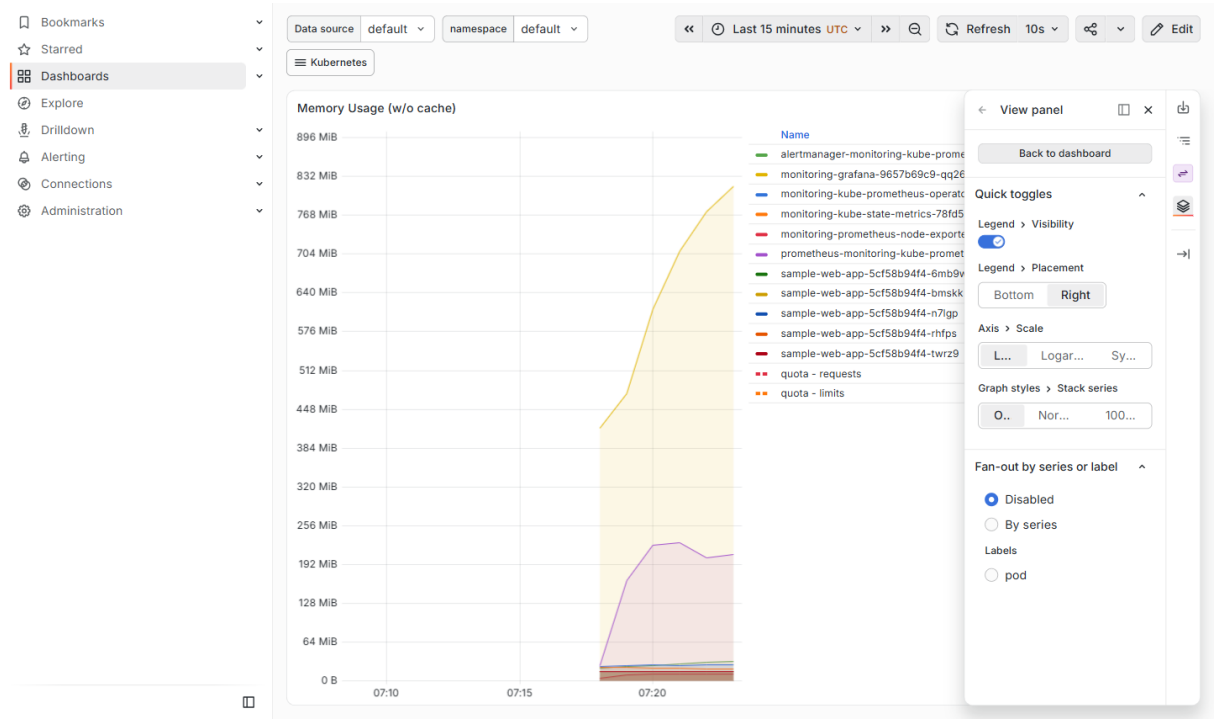


Fig. Live memory usage per pod in Grafana

7. Reflection Questions

R1. Pipeline Integration: How can ci-cd.yml run kubectl apply against the cluster after pushing a new image tag?

Add a deploy job that runs after build-and-push (needs: build-and-push). Store the cluster kubeconfig as a secret (KUBE_CONFIG), write it to ~/.kube/config, then apply the manifest and set the new image. A local Minikube cluster cannot be reached from GitHub-hosted runners, so this needs either a cloud cluster (EKS, GKE, AKS) or a self-hosted runner on the machine running Minikube (runs-on: self-hosted). Example:

```
  deploy:
    needs: build-and-push
    runs-on: self-hosted        # or ubuntu-latest for a cloud cluster
    steps:
      - uses: actions/checkout@v3
      - uses: azure/setup-kubectl@v3
      - name: Configure kubeconfig
        run: |
          mkdir -p ~/.kube
          echo "${{ secrets.KUBE_CONFIG }}" | base64 -d > ~/.kube/config
      - name: Deploy new image tag
        run: |
          kubectl apply -f deployment.yaml
          kubectl set image deployment/sample-web-app \
            web-container=${{ secrets.DOCKERHUB_USERNAME }}/sample-web-app:v1.0.${{
github.run_number }}
          kubectl rollout status deployment/sample-web-app --timeout=120s
```

The rollout status step fails the pipeline if the new pods never become ready, and a "kubectl rollout undo" step can be added for recovery. A GitOps tool such as Argo CD is another option: the pipeline commits the new tag to the manifest and Argo CD syncs the cluster.

R2. Resource Constraints: What happens when a container exceeds limits.memory? How would Grafana signal this?

Memory cannot be throttled the way CPU can. When a container goes over its memory limit (256Mi here), the kernel OOM killer ends the process. Kubernetes records the container as Terminated with reason OOMKilled (exit code 137) and restarts it according to restartPolicy. Repeated kills raise the restart count and the pod enters CrashLoopBackOff. Going over the CPU limit (250m) does not kill the container; it is throttled instead.

In Grafana with kube-prometheus-stack, the Compute Resources dashboards show memory working set climbing to the limit line, then dropping as the container restarts. The alert rules bundled with the stack (for example KubePodCrashLooping) fire through Alertmanager, and a custom rule can catch it directly:

```
kube_pod_container_status_last_terminated_reason{reason="OOMKilled"} == 1
or
```

```
container_memory_working_set_bytes / on(pod, container)
  kube_pod_container_resource_limits{resource="memory"} > 0.9
```

A Grafana alert on the memory panel can also notify by email or Slack when usage stays above 90% of the limit.

8. Conclusion

A declarative Deployment was created on Minikube and scaled from 2 to 5 replicas. A GitHub Actions pipeline built the Docker image on a push to main and pushed it to Docker Hub using repository secrets; the first run passed and the resulting v1.0.1 image was rolled out to the cluster. Cluster and pod health were monitored with metrics-server, the Kubernetes Dashboard, and Prometheus with Grafana.